CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 12: Independent Sets

Harvard SEAS - Spring 2026 *2026-03-09*

1 Announcements

- Anurag OH today after class (next one after Spring break).
- Midterm on Wed. Closed book. Bring black pen. Arrive early. Relax and get a good night's sleep!

Recommended Reading:

- Hesterberg–Vadhan Ch. 11.3-11.4, 13.1,13.3

2 Loose ends: Breadth-First Search

We discussed the following computational problem:

Input: A digraph $G = (V, E)$ and a source vertex $s \in V$

Output: The array $\texttt{dist}_s$ where for every $t \in V$, $\text{dist}_s[t] = \text{dist}_G(s, t)$

Computational Problem Single-Source Distances

With this, here is our first version of BFS.

```
BFS(G, s):
Input  : A digraph G = (V, E) and a source vertex s ∈ V
Output : The array dist_s[·] = dist_G(s, ·)
Initialize dist_s[t] = ∞ for all t ∈ V.;
S = F = {s};
dist_s[s] = 0;
foreach d = 1, ..., n − 1 do
    Let F = _____ ;
    For every v ∈ F, dist_s[v] = d;
    S = S ∪ F;
return dist_s
```

Algorithm 2.1: BFS for Single-Source Distances

Let's illustrate the behavior of BFS to the graph from Example **??** with $s = 0, t = 4$:

Q: What is happening at every iteration of the loop?

A:

Q: How do we prove correctness?

A:

Q: What is the runtime of the algorithm, in terms of the number of vertices n and the number of edges m?

A:

Q: How can we implement BFS faster?

A:

Putting all the above together, we obtain:

Theorem 2.1. `BFS`(G) *correctly solves* Single-Source Distances *and can be implemented in time* $O(n+m)$*, where* n *is the number of vertices in* G *and* m *is the number of edges in* G*.*

Implementation details.

- In practice, F is stored as a queue and updates are done one vertex u at a time rather than as a 'batch'.
- The bitvector S is redundant given that we are maintaining $\texttt{dist}_s$.

3 Finding the Paths

Q: How to actually find shortest *path* from s to t, not just the distances?

A:

Input: A digraph $G = (V, E)$ and a source vertex $s \in V$
Queries: For any query vertex t, return a shortest path from s to t (if one exists).

Abstract Data Type Single-Source Shortest Paths

Theorem 3.1. Single-Source Shortest Paths *has a data structure that, for digraphs with* n *vertices and* m *edges, has preprocessing time* $O(n+m)$ *and the time to answer a query* t *is* $O(\mathrm{dist}_G(s,t))$*.*

Given this, we deduce an algorithm for the Shortest Walk problem we started with, by running the preprocessor and then querying the data structure once with our given destination vertex t:

Corollary 3.2. Shortest Walk *(equivalently,* Shortest Path*) can be solved in time* $O(n+m)$*.*

It is natural and very useful (e.g. in Google maps) to have data structures for variants of Shortest Path on *dynamic graphs*. There is a rich collection of methods for dynamic graph data structures, which are beyond the scope of this course.

Another extension of BFS is to handle *weighted graphs*. This is Djikstra's algorithm, and is covered in CS 1240.

4 Independent Sets: Definition

Recall the Interval Scheduling–Decision problem introduced in earlier in the course:

Input: A collection of intervals $[a_0, b_0], \ldots, [a_{n-1}, b_{n-1}]$, where for all i, $a_i, b_i \in \mathbb{Q}$ and $a_i \leq b_i$

Output: YES if the intervals are disjoint from each other (for all $i \neq j$, $[a_i, b_i] \cap [a_j, b_j] = \emptyset$)
NO otherwise

Computational Problem Interval Scheduling–Decision

(Here we have modified the problem to have the endpoints a_i, b_i be rational numbers, since our computational models cannot represent arbitrary real numbers.)

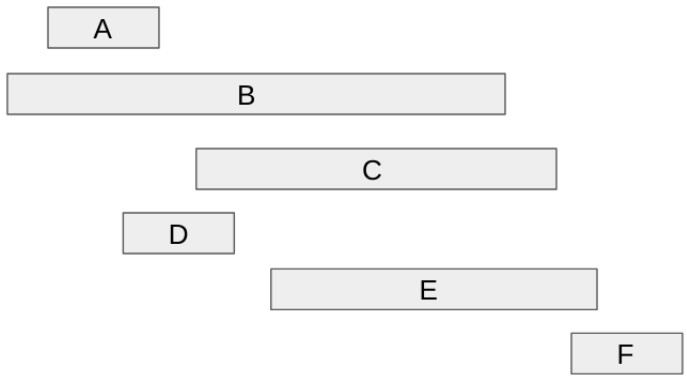
We saw that we could solve this problem in time $O(n \log n)$ by reduction to Sorting. However, if the answer is NO, we might be satisfied by trying to schedule *as many* intervals *as possible*:

Input: A collection of intervals $[a_0, b_0], \ldots, [a_{n-1}, b_{n-1}]$, where each $a_i, b_i \in \mathbb{Q}$ and $a_i \leq b_i$

Output: A maximum-size subset $S \subseteq [n]$ such that $\forall i \neq j \in S$, $[a_i, b_i] \cap [a_j, b_j] = \emptyset$.

Computational Problem Interval Scheduling–Optimization

Let's see how we can reduce this to a graph-theoretic problem. For example, consider the following set of intervals:



To model this graph-theoretically, we can

A set of intervals that do not intersect correspond to a set of vertices that do not have edges between them.

Definition 4.1. Let $G = (V, E)$ be a graph. An *independent set* in G is a subset $S \subseteq V$ such that there are no edges entirely in S. That is, $\{u, v\} \in E$ implies that $u \notin S$ or $v \notin S$.

Finding the largest set of conflict-free intervals thus amounts to finding the largest independent set in the corresponding conflict graph.

Input: A graph $G = (V, E)$
Output: An independent set $S \subseteq V$ in G of maximum size

Computational Problem Independent Set

Another example of what is captured by the Independent Set problem is coming up with as long an invitation list as possible for a party with the constraint that all attendees get along with each other.

5 Greedy Algorithms

An exhaustive search algorithm for Independent Set is:

There is an algorithm that takes time

$$O\left((n+m)\binom{n}{n/3}\right) = O(1.89^n),$$

by taking advantage of graph coloring (we will cover Coloring on Mar 23). This will be the content of next SRE on Mar 25.

Unfortunately, the fastest known algorithms still run in exponential time. For efficient algorithms to get large (but likely not the largest) independent sets, it is natural to try a greedy strategy. Such an approach is described in Algorithm 5.1.

```
GreedyIndSet(G):
Input  : A graph G = (V, E)
Output : A "large" independent set in G
Choose an ordering v_0, v_1, v_2, ..., v_{n-1} of V;
S = ∅;
foreach i = 0 to n − 1 do
    ____________________________________________;
return S
```

Algorithm 5.1: `GreedyIndSet()`

How large is the independent set it finds? We can prove only fairly weak bounds on its performance in general:

Theorem 5.1. *For every graph G with n vertices and m edges, and every vertex ordering,* `GreedyIndSet(G)` *outputs an independent set of size at least $n/(\deg_{max}+1)$, where $\deg_{max}$ is the maximum vertex degree in G.*

Proof sketch.

□

The guarantee of Theorem 5.1 can be quite weak compared to the size of the largest independent set in G. However, we can do much better for Interval Scheduling–Optimization, and find the true largest independent set. The point is that the conflict graphs that arise from collections of intervals are much more structured than arbitrary graphs.

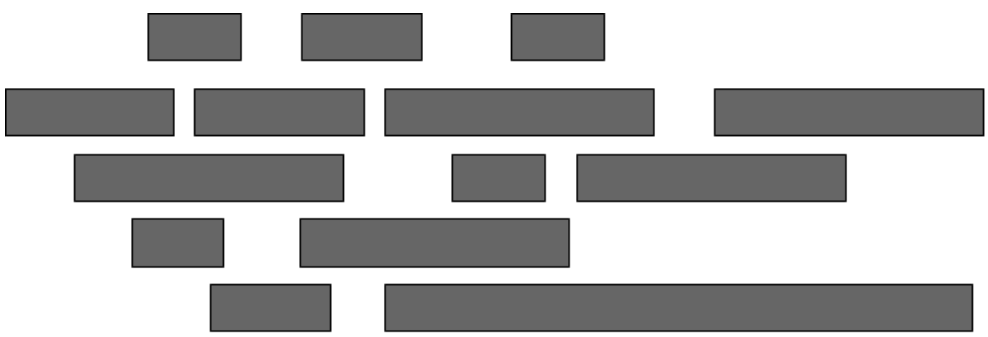
Specifically, a nearly linear-time greedy algorithm for INTERVAL SCHEDULING–OPTIMIZATION is given in Algorithm 5.2.

```
GreedyIntervalScheduling(x):
Input  : A list x of n intervals [a, b], with a, b ∈ ℚ
Output : A "large" subset of the input intervals that are disjoint
         from each other
Sort the intervals with                    ;
S = ∅;
foreach i = 0 to n − 1 do
    |                                      ;
    | return S
```

Algorithm 5.2: `GreedyIntervalScheduling()`

Theorem 5.2. *For every input list x of intervals,* `GreedyIntervalScheduling(x)` *finds an optimal solution to* INTERVAL SCHEDULING–OPTIMIZATION *in time $O(n \log n)$.*

Proof. For correctness, we will show that for any INTERVAL SCHEDULING–OPTIMIZATION instance (such as the one shown in black in below) and any solution $S^* = \{i_0^* \leq i_1^* \leq \ldots \leq i_{k^*-1}^*\}$ (where $i_j^* \in S^*$ means that we include the interval $[a'_{i_j^*}, b'_{i_j^*}]$), we can modify S^* into the greedy solution $S = \{i_0, i_1, \ldots, i_{k-1}\}$ by "smushing it left."



Induction Hypothesis. Formally, we can prove the following by induction on $j = 0, 1, \ldots, k^* - 1$:

1. The greedy solution contains at least $j + 1$ intervals.

2. $b'_{i_j} \leq b'_{i^*_j}$, i.e. the jth interval selected by the greedy algorithm ends no later than the jth interval in the optimal solution ("greedy stays ahead").

Setting $j = k^* - 1$ in Item 1, we see that `GreedyIntervalScheduling` finds a solution containing at least k^* intervals. That is, for every independent set S^*, `GreedyIntervalScheduling` finds an independent set with at least as many intervals as S^*. Thus, `GreedyIntervalScheduling` finds an optimal solution.

Runtime.

$\square$